GOLD: Selection (Wed Sep 2)

Review: Control Flow

Sequential Execution

- By default, a list of statements

```
Statement 1;
Statement 2;
…
Statement N;
```

is executed one after another in sequence, unless execution of a statement causes an error ("exception") and the "system" terminates the program

- We say, "Control flows sequentially."

Control Structures

- Sequential is the default control flow
- Sequential is fast and common, but alone is too limiting to express interesting programs
- Keywords that change the default control flow:

`if`	`break`
`switch`	`continue`
`while`	`return`
`for`	

- Also, control flow can jump to start of a called method, then returns

boolean Operations

Combining conditions: `&&`, `||`, `!`

A condition (used in `if` statements) is a `boolean`. There are only two values: either `true` or `false`.

We have a few operators that combine them:

Operator	Name	True when...
`a && b`	AND	**both** are true
`a || b`	OR	**at least one** is true
`!a`	NOT	`a` is false

```
true  && false   // false
true  || false   // true
!true            // false
```

Full truth tables:

a	**b**	**a && b**	**a || b**
`true`	`true`	`true`	`true`
`true`	`false`	`false`	`true`
`false`	`true`	`false`	`true`
`false`	`false`	`false`	`false`

and for NOT:

a	**!a**
`true`	`false`
`false`	`true`

What's wrong with this code?

```
void main() {
    int qty = Integer.parseInt(IO.readln("How many? "));
    if (qty >= 1 || qty <= 10) {
        IO.println("Order accepted: " + qty);
    } else {
        IO.println("Please order 1 to 10.");
    }
}
```

Let's break this down:

- First, what is it trying to do? The two branches will give us a clue.
- Second, if we suspect a bug, how might we find it?
    - **Run** the code with different inputs. This is easy here because the user is asked for input.
    - **Trace the code by hand** line-by-line for different hypothetical inputs
- After making any changes, test the code again
-

Pause and predict

Q1. Which of the following uses of boolean operators will translate this sentence? (Select all that apply — assume the variables and functions mentioned exist.)

"If it is a weekday and I'm not on vacation, then I will get up early"

- **A.** `if (!onVacation && isWeekday) getUpEarly();`
- **B.** `if (!(onVacation && isWeekday)) getUpEarly();`
- **C.** `if (!onVacation and isWeekday) getUpEarly();`
- **D.** `if (!(onVacation and isWeekday)) getUpEarly();`

Q2. "If a customer orders a burger and either fries or a drink, give them the combo discount." (Assume `orderedBurger`, `orderedFries`, `orderedDrink` are `booleans` and `giveDiscount()` exists.)

- **A.** `if (orderedBurger && orderedFries || orderedDrink) { giveDiscount(); }`
- **B.** `if (orderedBurger && (orderedFries || orderedDrink)) { giveDiscount(); }`
- **C.** `if (orderedBurger || orderedFries && orderedDrink) { giveDiscount(); }`
- **D.** `if ((orderedBurger && orderedFries) || orderedDrink) { giveDiscount(); }`

Poll Reveal: boolean Examples

Question 1

Which of the follow uses of Boolean operators will translate the following sentence (assuming the variables and functions mentioned exist)?

"If it is a weekday and I'm not on vacation, then I will get up early"

- [ ] `if (!onVacation && isWeekday) getUpEarly();`
- [ ] `if (!(onVacation && isWeekday)) getUpEarly();`
- [ ] `if (!onVacation and isWeekday) getUpEarly();`
- [ ] `if (!(onVacation and isWeekday)) getUpEarly();`

Question 2

"If a customer orders a burger and either fries or a drink, give them the combo discount."

(Assume `orderedBurger`, `orderedFries`, `orderedDrink` are `booleans` and `giveDiscount()` exists.)

- ( ) `if (orderedBurger && orderedFries || orderedDrink) { giveDiscount(); }`
- ( ) `if (orderedBurger && (orderedFries || orderedDrink)) { giveDiscount(); }`

```
if (orderedBurger || orderedFries && orderedDrink) { giveDiscount(); }
```

```
if ((orderedBurger && orderedFries) || orderedDrink) { giveDiscount(); }
```

If Statement Syntax

The basic

```
if (boolean-expression) {
  statement-to-execute-if-Boolean-is-true;
}

if (boolean-expression) {
  statement-to-execute-if-Boolean-is-true;
} else {
  statement-to-execute-if-Boolean-is-false;
}
```

The braces `{` and `}` are used to group a block of statements together.

This way Java knows where the `if` and `else` branches end.

As a shorthand, if there is **only one statement** in the block, we can omit the braces:

```
if (boolean-expression)
  statement-to-execute-if-Boolean-is-true;

if (boolean-expression)
  statement-to-execute-if-Boolean-is-true;
else
  statement-to-execute-if-Boolean-is-false;
```



Good style for CS 18000: **always use the braces**, even for a single statement. You'll see the braceless shorthand in the wild, but this increases the chances of making a mistake later.

For **three or more** possibilities, chain the conditions:

```
if (creditsEarned >= 90) {
    IO.println("Senior");
} else if (creditsEarned >= 60) {
    IO.println("Junior");
} else if (creditsEarned >= 30) {
    IO.println("Sophomore");
} else {
    IO.println("Freshman");
}
```

Conditions are tested top to bottom; the **first** true one wins and the rest are skipped. The order of the rungs matters.

Strings Are Characters

From characters to text: `String`

A **`String` is a sequence of `char`s**, numbered from **0**:

```
index:   0   1   2   3   4   5
         B   U   R   G   E   R
```

```
void main() {
    String s = "BURGER";
    s.length();    // 6     how many characters
    IO.println(s.charAt(0));   // 'B'   the char at a position
    IO.println(s.charAt(2));   // 'R'

    // IO.println(s.charAt(-1)); NO -- out of bounds
}
```

Valid positions are **0 to length − 1**.

`s.charAt(6)` **crashes** because there is no position 6:

```
Exception in thread "main" java.lang.StringIndexOutOfBoundsException: Index 6 out of b
```

A **run-time** error is a lot of text, but it is readable:

- It tells you which index and what length.

String Operations

Looking inside a `String`

Here are several basic String operations we'll use again and again:

```
void main() {
    String s = "BURGER";
    IO.println(s.substring(1, 4));   // "URG"   index 1 up to — but NOT including — 4
    IO.println(s.substring(3));      // "GER"   from index 3 to the end
    IO.println(s.indexOf('R'));      // 2       position of the FIRST match
    IO.println(s.indexOf('Z'));      // -1      not found
    IO.println(s.contains("URG"));   // true
}
```

`substring(begin, end)` includes `begin`, excludes `end`

- the same "up to but not including" rule as `charAt`'s last index.

To compare text, use **`.equals`**, never `==`:

```
s.equals("BURGER")              // true
s.equalsIgnoreCase("burger")    // true
```

Reference: String

Reference: common `String` methods

Reference slide: look up as needed

All of these methods must be called following a string (variable) and a `.`

Example: `s.charAt(1)`

Method	Returns	Example on `"BURGER"`
`length()`	`int`	`6`
`charAt(i)`	`char`	`charAt(0)` → `'B'`
`substring(b, e)`	`String`	`substring(1,4)` → `"URG"`
`substring(b)`	`String`	`substring(3)` → `"GER"`
`indexOf(x)`	`int`	`indexOf('R')` → 2; missing → `-1`
`contains(x)`	`boolean`	`contains("URG")` → `true`
`startsWith(x)` / `endsWith(x)`	`boolean`	`startsWith("BUR")` → `true`
`toUpperCase()` / `toLowerCase()`	`String`	`toLowerCase()` → `"burger"`
`replace(a, b)`	`String`	`replace('R','X')` → `"BUXGEX"`
`trim()` / `strip()`	`String`	removes surrounding spaces
`equals(x)` / `equalsIgnoreCase(x)`	`boolean`	text comparison (not `==`)
`isEmpty()`	`boolean`	`"".isEmpty()` → `true`

Short-Circuit Operators

`&&` and `||` are lazy

Java stops evaluating an expression with `&&` or `||` as soon as the answer is certain:

- `a && b` : if `a` is **false**, it skips `b` (already false).
- `a || b` : if `a` is **true**, it skips `b` (already true).

This lets you **guard** a risky operation. For example: `charAt` crashes on an empty string, and we want to avoid that.

This is safe:

```
// Get a code word from the user; check if it begins with a 'B'
void main() {
    String text = IO.readln("Enter a code word: ");

    if (text.length() > 0) {
        if (text.charAt(0) == 'B') {
            IO.println("...");
        }
    }

    // shorter:
    if (text.length() > 0 && text.charAt(0) == 'B') {
        IO.println("Starts with B");
    }
    IO.println("Goodbye!");
}
```

Pause and predict (poll)

Q. In the shorter form above, the user just presses Enter, which means the input is the empty string `""`. What happens?

- **A.** Crash: `StringIndexOutOfBoundsException`
- **B.** Prints `Goodbye!` only
- **C.** Prints `Starts with B` then `Goodbye!`
- **D.** Prints nothing at all

Poll Reveal: Just Enter

Question

The user just presses Enter, which means the input is the empty string `""`:

```
if (text.length() > 0 && text.charAt(0) == 'B') {
    IO.println("Starts with B");
}
IO.println("Goodbye!");
```

What happens?

- Crash: `StringIndexOutOfBoundsException`
- Prints `Goodbye!` only
- Prints `Starts with B` then `Goodbye!`
- Prints nothing at all

Demo: The Yes/No Gate

Demo: the yes/no gate

Problem: Ask the user a yes/no question and act only on a clear “yes”. Let's write code that works on "yes"/"Yes"/"YES" etc.

Plan:

1. Read the answer with `IO.readln`.
2. If the answer is empty, it is not a yes; don’t even look at its letters.
3. Otherwise look at the first character: `'y'` or `'Y'` counts.

Code: (constructed during class)

```
void main() {
    String ans = IO.readln("Delete all files? (yes/no) ");

    if (ans.length() > 0) {
        char first = ans.charAt(0);
        if (first == 'y' || first == 'Y') {
            IO.println("Deleting... (not really)");
        }
    }
    IO.println("Cancelled.");   // hmm... When does this line print?
}
```

Two things to fix and discuss:

- The `Cancelled.` line prints **every** time: it needs an `else`. But where do the braces go?

- The nested `if`s can collapse into one guard by using the short-circuit shape:

```
if (ans.length() > 0 && (ans.charAt(0) == 'y' || ans.charAt(0) == 'Y')) {
    IO.println("Deleting... (not really)");
} else {
    IO.println("Cancelled.");
}
```

▶ Expand

Immutability

Strings don't change: immutability

A `String`'s characters are fixed the moment it's created.

In technical terms we say: Strings are **immutable**.

- There's no way to edit a character or extend a String in place.
- Every String operation hands you back a **brand-new** `String` and leaves the original alone:

```
String name = "ada";
name.toUpperCase();        // returns a new "ADA", then discards it
IO.println(name);         // still prints: ada
```



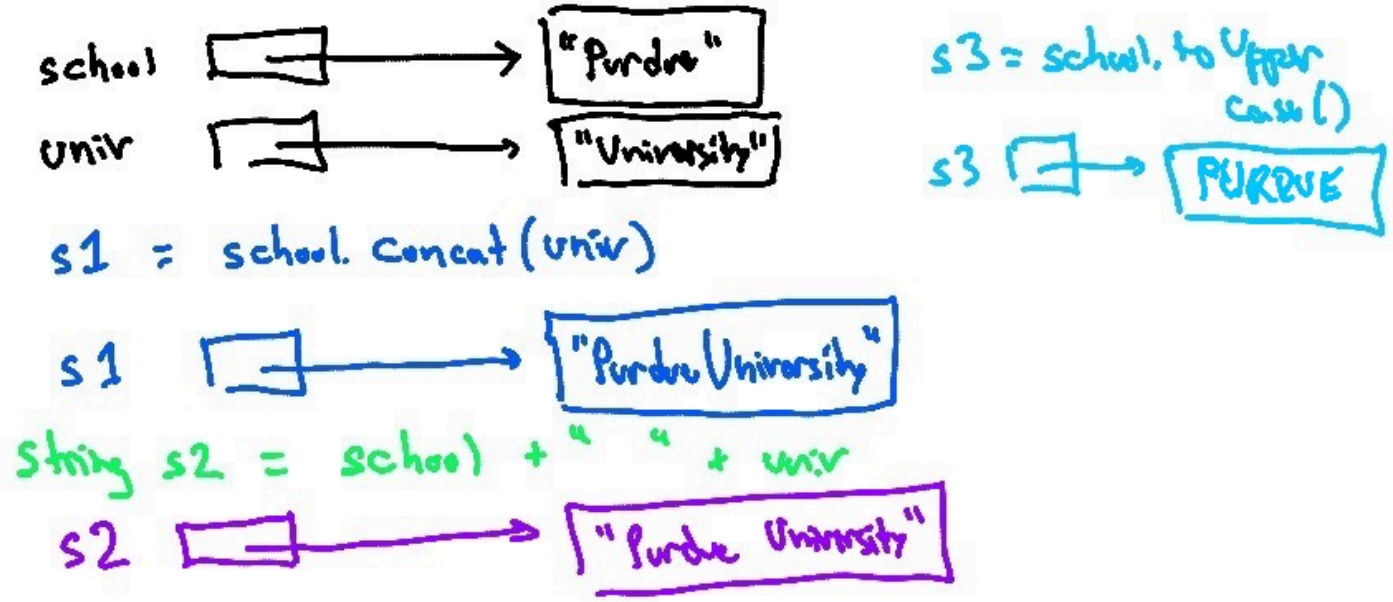
Immutability is a special property specific to Strings. When we meet other reference types and objects later, these will typically **not** be immutable.

Here's a larger example:

```
void main() {
    String school = "Purdue";
    String univ = "University";

    String s1 = school.concat(univ);   // "PurdueUniversity"
    String s2 = school + " " + univ;   // "Purdue University"
    String s3 = school.toUpperCase();  // "PURDUE"
}
```

Before we add `println` statements above, let's trace out what this does in detail:


school
univ
"Purdue"
"University"
s3 = school.toUpperCase()
s3
PURDUE
s1 = school.concat(univ)
s1
"Purdue University"
String s2 = school + " " + univ
s2
"Purdue University"

▶ Expand

Reassignment vs Mutation

Reassignment vs. changing in place

If you can't edit a String's letters, how do you "update" one?

You must **reassign** the variable using `=`. Remember: `=`-is-an-action!

```
void main() {
    String name = "purdue";
    name = name.toUpperCase();   // compute "PURDUE", then store it back in name
    IO.println(name);            // PURDUE
}
```

Nothing edited the original `"purdue"`. Java made a **new** String `"PURDUE"` and pointed `name` at it.

For reference types, such as String, there are two different meanings of change that we need to keep separate:

- **Changing in place:** editing the existing letters. *This is not possible for Strings.*
- **Reassignment:** compute a new String and store it back to the same variable name.
    - This is what we actually do.
    - The variable is a box containing a *different* address.

This distinction does not exist for primitive types (`int`, `double`, `char`, `boolean` etc) because the number is stored directly at the place in memory where a variable name refers.

There is no indirection for primitives.

Pause and predict (poll)

Q. What will this print?

```
String greeting = "hello";
greeting.replace("l", "L");
IO.println(greeting);
```

- **A.** nothing (blank)
- **B.** hello
- **C.** heLLo
- **D.** heLlo
- **E.** something else

Poll Reveal: replace

Question

What will this print?

```
String greeting = "hello";
greeting.replace("l", "L");
IO.println(greeting);
```

- nothing (blank)
- hello
- heLLo
- heLlo
- something else

Demo: Capitalize the Name

Demo: capitalize the name

Problem: Read a name and make sure it starts with a capital letter. If it's not broken, don't fix it!

Remember, Strings don’t change in place, so the fix is a **reassignment**.

Steps:

1. Ask the user for their name.
2. If the name starts with 'a', 'b', ..., 'z':
    1. Create a new String with the corresponding capital letter and the rest of the name
    2. Reassign it to the name variable

Code:

```
void main() {
    String name = IO.readln("Name? ");

    IO.println("Before: " + name);
    if (name.length() > 0 && name.charAt(0) >= 'a' && name.charAt(0) <= 'z') {
        name = name.substring(0, 1).toUpperCase() + name.substring(1);
    }
    IO.println("After: " + name);
}
```

Why each piece of the condition:

- `name.length() > 0` is the guard; an empty name has no `charAt(0)` to look at.
- `>= 'a' && <= 'z'` : char comparisons; already-capitalized names (and “names” starting with a digit) skip the fix.
- `substring(0, 1).toUpperCase() + substring(1)` builds a **new** String; `name =` points the variable at it.

Take Home: Tutor Wages

Your friend Alice has now opened her own tutoring company and is trying to lure you away from Bob's company.

If you work

- **more than 20 hours** at Alice's tutoring, she will pay you:
- $18/hour for the first 20 hours
- plus $25/hour for all hours above 20.
- **Otherwise**, she pays you $16/hour.

Your task is to implement a method

```
double computePay(double hours)
```

that returns the correct pay.

Press **Test** to run the tests. Pay special attention to the boundary between the two rules.